

Exercise 2: Implementing the Factory Method Pattern

Scenario

You are developing a document management system that needs to create different types of documents (Word, PDF, Excel). The Factory Method Pattern is used to delegate the creation of documents to subclasses, keeping the client code decoupled from concrete implementations.

What is the Factory Method Pattern?

The **Factory Method Pattern** is a Creational Design Pattern that defines an interface for creating objects but lets subclasses decide which class to instantiate. It promotes the Open/Closed Principle — open for extension, closed for modification.

Component	Role	In This Exercise
Document (interface)	Product Interface	open(), save(), close(), getType()
WordDocument	Concrete Product	Handles .docx files
PdfDocument	Concrete Product	Handles .pdf files
ExcelDocument	Concrete Product	Handles .xlsx files
DocumentFactory	Abstract Creator	Declares createDocument()
WordDocumentFactory	Concrete Creator	Returns new WordDocument()
PdfDocumentFactory	Concrete Creator	Returns new PdfDocument()
ExcelDocumentFactory	Concrete Creator	Returns new ExcelDocument()

Project Structure: FactoryMethodPatternExample

```
FactoryMethodPatternExample/  
  Document.java          <-- Product Interface  
  WordDocument.java      <-- Concrete Product  
  PdfDocument.java       <-- Concrete Product  
  ExcelDocument.java     <-- Concrete Product  
  DocumentFactory.java   <-- Abstract Creator (Factory)  
  WordDocumentFactory.java <-- Concrete Creator  
  PdfDocumentFactory.java <-- Concrete Creator  
  ExcelDocumentFactory.java <-- Concrete Creator  
  FactoryMethodTest.java <-- Test / Client
```

Step 1: Document Interface

Defines the contract for all document types.

```
/**  
 * Document.java - Base interface for all document types  
 */  
public interface Document {  
    void open();  
    void save();
```

```

        void close();
        String getType();
    }

```

Step 2: Concrete Document Classes

Each class implements the Document interface for a specific file type.

```

// WordDocument.java
public class WordDocument implements Document {
    @Override
    public void open() { System.out.println("[WordDocument] Opening Word document (.docx)..."); }
    @Override
    public void save() { System.out.println("[WordDocument] Saving Word document (.docx)..."); }
    @Override
    public void close() { System.out.println("[WordDocument] Closing Word document."); }
    @Override
    public String getType() { return "Word Document (.docx)"; }
}

// PdfDocument.java
public class PdfDocument implements Document {
    @Override
    public void open() { System.out.println("[PdfDocument] Opening PDF document (.pdf)..."); }
    @Override
    public void save() { System.out.println("[PdfDocument] Saving PDF document (.pdf)..."); }
    @Override
    public void close() { System.out.println("[PdfDocument] Closing PDF document."); }
    @Override
    public String getType() { return "PDF Document (.pdf)"; }
}

// ExcelDocument.java
public class ExcelDocument implements Document {
    @Override
    public void open() { System.out.println("[ExcelDocument] Opening Excel spreadsheet (.xlsx)..."); }
    @Override
    public void save() { System.out.println("[ExcelDocument] Saving Excel spreadsheet (.xlsx)..."); }
    @Override
    public void close() { System.out.println("[ExcelDocument] Closing Excel spreadsheet."); }
    @Override
    public String getType() { return "Excel Document (.xlsx)"; }
}

```

Step 3: Abstract DocumentFactory

Declares the factory method createDocument() and a template method openDocument().

```

/**
 * DocumentFactory.java - Abstract Factory with the Factory Method
 */
public abstract class DocumentFactory {

    // Factory Method - overridden by each concrete subclass
    public abstract Document createDocument();

    // Template method that uses the factory method
    public void openDocument() {
        Document doc = createDocument();
        System.out.println("Created: " + doc.getType());
        doc.open();
        doc.save();
    }
}

```

```

        doc.close();
    }
}

```

Step 4: Concrete Factory Classes

Each factory decides which Document subclass to instantiate.

```

// WordDocumentFactory.java
public class WordDocumentFactory extends DocumentFactory {
    @Override
    public Document createDocument() { return new WordDocument(); }
}

// PdfDocumentFactory.java
public class PdfDocumentFactory extends DocumentFactory {
    @Override
    public Document createDocument() { return new PdfDocument(); }
}

// ExcelDocumentFactory.java
public class ExcelDocumentFactory extends DocumentFactory {
    @Override
    public Document createDocument() { return new ExcelDocument(); }
}

```

Step 5: Test Class — FactoryMethodTest.java

Demonstrates creation of different document types using factory methods.

```

/**
 * FactoryMethodTest.java - Client / Test class
 */
public class FactoryMethodTest {

    public static void main(String[] args) {
        System.out.println("=== Factory Method Pattern Test ===\n");

        // Create Word Document via its factory
        System.out.println("--- Test 1: Word Document ---");
        DocumentFactory wordFactory = new WordDocumentFactory();
        wordFactory.openDocument();

        // Create PDF Document via its factory
        System.out.println("\n--- Test 2: PDF Document ---");
        DocumentFactory pdfFactory = new PdfDocumentFactory();
        pdfFactory.openDocument();

        // Create Excel Document via its factory
        System.out.println("\n--- Test 3: Excel Document ---");
        DocumentFactory excelFactory = new ExcelDocumentFactory();
        excelFactory.openDocument();

        // Polymorphic usage
        System.out.println("\n--- Test 4: Polymorphic Usage ---");
        DocumentFactory[] factories = {
            new WordDocumentFactory(),
            new PdfDocumentFactory(),
            new ExcelDocumentFactory()
        };
        for (DocumentFactory factory : factories) {
            Document doc = factory.createDocument();

```

```

        System.out.println("Document type created: " + doc.getType());
    }

    System.out.println("\n SUCCESS: Factory Method Pattern correctly creates" +
        " different document types!");
}
}

```

Expected Output

```

=== Factory Method Pattern Test ===

--- Test 1: Word Document ---
Created: Word Document (.docx)
[WordDocument] Opening Word document (.docx)...
[WordDocument] Saving Word document (.docx)...
[WordDocument] Closing Word document.

--- Test 2: PDF Document ---
Created: PDF Document (.pdf)
[PdfDocument] Opening PDF document (.pdf)...
[PdfDocument] Saving PDF document (.pdf)...
[PdfDocument] Closing PDF document.

--- Test 3: Excel Document ---
Created: Excel Document (.xlsx)
[ExcelDocument] Opening Excel spreadsheet (.xlsx)...
[ExcelDocument] Saving Excel spreadsheet (.xlsx)...
[ExcelDocument] Closing Excel spreadsheet.

--- Test 4: Polymorphic Usage ---
Document type created: Word Document (.docx)
Document type created: PDF Document (.pdf)
Document type created: Excel Document (.xlsx)

SUCCESS: Factory Method Pattern correctly creates different document types!

```

Key Design Principles Applied

Open/Closed Principle: New document types can be added by creating a new Document subclass and a matching factory — no existing code needs modification.

Single Responsibility: Each factory class has one responsibility: creating its specific document type.

Dependency Inversion: The client (FactoryMethodTest) depends on the abstract DocumentFactory, not on concrete classes.

Polymorphism: DocumentFactory[] array holds different factory types and calls openDocument() polymorphically.